

**DEV**[Create account](#)**Serif COLAKEL**

Posted on Oct 6, 2024 • Edited on Oct 7, 2024



Creating and Using VS Code Snippets: A Beginner's Guide

#webdev #javascript #programming #vscode



image no longer exists

Visual Studio Code (VS Code) offers a powerful feature called snippets that allows you to insert blocks of code quickly by typing a short prefix and hitting Tab. Snippets help you automate repetitive tasks, maintain consistency in your code, and save time. In this article, we'll explore how to create your own snippets, use dynamic variables, and see examples for popular JavaScript frameworks like React, Angular, Vue, and Node.js.

Understanding Variables in Snippets

To create or edit your own snippets, select **Configure User Snippets** under **Code > Settings**, and then select the language (by language identifier) for which the snippets should appear. You can also create global snippets that work across all languages.

Here's a basic example of a `for` loop snippet in JavaScript:

```
{
  "For Loop": {
    "prefix": "for",
    "body": [
      "for (let ${1:i} = 0; ${1} < ${2:array}.length; ${1}++) {",
      "  const ${3:element} = ${2[${1}]};",
      "  ${0}",
      "}"
    ],
    "description": "Standard for loop"
  }
}
```

In the example above:

- "For Loop" is the snippet name. It is displayed via IntelliSense if no `description` is provided.
- `prefix` defines one or more trigger words that display the snippet in IntelliSense. Substring matching is performed on prefixes, so in this case, "fc" could match "for-const".
- `body` is one or more lines of content, which will be joined as multiple lines upon insertion. Newlines and embedded tabs will be formatted according to the context in which the snippet is inserted.
- `description` is an optional `description` of the snippet displayed by IntelliSense.

javascript.json (JavaScript)
markdown.json (Markdown)

New Global Snippets file...

bat (Batch)

c (C)

clojure (Clojure)

coffeescript (CoffeeScript)

cpp (C++)

csharp (C#)

css (CSS)

Existing Snippets

New Snippets

Variables make snippets more dynamic and contextual. Instead of hardcoding values, variables allow you to insert dynamic content such as file names, the current date, or even random values like UUIDs. Here's how variables work:

- `$name`: Inserts the value of the variable `name`.
- `${name}`: Inserts `default` if the variable is not defined or empty.

Common Variables [Reference](#)

Here are some commonly used variables that can make your snippets more powerful:

- `TM_FILENAME`: The name of the current file, including its extension.
- `TM_FILENAME_BASE`: The file name without its extension.
- `TM_SELECTED_TEXT`: The currently selected text, or an empty string if none is selected.
- `CURRENT_YEAR`: The current year.
- `UUID`: A randomly generated UUID (Version 4).

Example Snippet: Console Log with Dynamic Variables

Let's start with a simple JavaScript snippet that inserts a `console.log` statement. It will use the file name and line number dynamically:

```
{  
  "Console Log": {  
    "prefix": "clog",  
    "body": [  
      "console.log('Debugging ${TM_FILENAME} at line ${TM_LINE_NUMBER}:', ${TM_SELECTED_TEXT})"    ]  
  }  
}
```

```
}
```

Explanation:

- `TM_FILENAME`: Inserts the name of the file where the snippet is triggered.
- `TM_LINE_NUMBER`: Inserts the current line number.
- `TM_SELECTED_TEXT`: Logs any selected text, making it useful for debugging.

Example Snippet: Using Variables

Here's an example of a basic snippet that uses several variables. Let's create a "Print Debug" snippet for JavaScript, which automatically includes file and line information:

```
{
  "Print Debug": {
    "prefix": "debug",
    "body": [
      "console.log('Debugging ${TM_FILENAME} at line ${TM_LINE_NUMBER}:');",
      "console.log('${TM_SELECTED_TEXT}');"
    ],
    "description": "Inserts a debug log with file name and line number"
  }
}
```

Explanation:

- `TM_FILENAME`: Inserts the name of the file where the snippet is triggered.
- `TM_LINE_NUMBER`: Inserts the current line number.
- `TM_SELECTED_TEXT`: Logs any selected text, making it useful for debugging.

Example Snippet: Adding Date and Time

You can also insert the current date and time, which is especially useful for comments or logging.

```
{
  "Current Date": {
    "prefix": "timestamp",
    "body": [
```

```
},
  "description": "Inserts a timestamp with date and time"
}
}
```

Explanation:

- `CURRENT_DAY_NAME`: The full name of the current day (e.g., Monday).
- `CURRENT_MONTH_NAME`: The full name of the current month (e.g., January).
- `CURRENT_DATE`: The current date (e.g., 1).
- `CURRENT_YEAR`: The current year (e.g., 2022).
- `CURRENT_HOUR`: The current hour in 24-hour format (e.g., 14).
- `CURRENT_MINUTE`: The current minute (e.g., 30).
- `CURRENT_SECOND`: The current second (e.g., 45).
- `${CURRENT_HOUR}:${CURRENT_MINUTE}:${CURRENT_SECOND}`: Inserts the current time in hours, minutes, and seconds.
- `CURRENT_TIMEZONE_OFFSET`: Inserts the time zone offset (e.g., "-07:00").

Example Snippet: Using Random Values

You can insert random values into your snippets, such as UUIDs or random numbers. This is useful for generating placeholder data or unique identifiers.

```
{
  "Generate UUID": {
    "prefix": "uuid",
    "body": [
      "const uuid = '${UUID}';",
      "console.log('Generated UUID:', uuid);"
    ],
    "description": "Inserts a random UUID"
  },
  "Wrap with Function": {
    "prefix": "wrapfunc",
    "body": [
      "function ${1:functionName}() {",
      "  ${2:// your code here}",
      "}",
      "${3:/* selected text */}"
    ]
  }
}
```

```
},  
"Generate crypto.randomUUID": {  
  "prefix": "randomUUID",  
  "body": ["crypto.randomUUID();"],  
  "description": "Inserts a random UUID using Node.js"  
}  
}
```

Explanation:

- `UUID`: Inserts a randomly generated UUID (Version 4).
- `wrapfunc`: Wraps the selected text in a function.
- `randomUUID`: Inserts a random UUID using Node.js.

Example Snippet: Commenting Code

When working in different programming languages, you often need to insert comments. VS Code allows you to insert the correct block or line comment based on the current language.

```
{  
  "Block Comment": {  
    "prefix": "block",  
    "body": ["/**", " * ${TM_SELECTED_TEXT}", " */"],  
    "description": "Inserts a block comment"  
  },  
  "Line Comment": {  
    "prefix": "line",  
    "body": ["// ${TM_SELECTED_TEXT}"],  
    "description": "Inserts a line comment"  
  }  
}
```

Explanation:

- `block`: Inserts a block comment.
- `line`: Inserts a line comment.

Example Snippet: React useReducer Hook

```
{
  "Dynamic useReducer Hook": {
    "prefix": "useReducerDynamic",
    "body": [
      "type ${1:StateType} = { ${2:key: value} };",
      "",
      "type ${3:ActionType} =",
      "  | { type: '${4:ACTION_TYPE_1}'; ${5:payload?: value} }",
      "  | { type: '${6:ACTION_TYPE_2}'; ${7:payload?: value} };",
      "",
      "const ${8:reducerName} = (state: ${1:StateType}, action: ${3:ActionType}): ${1:StateType} => {",
      "  switch (action.type) {",
      "    case '${4:ACTION_TYPE_1}':",
      "      return { ...state, ${9:updatedState1} };",
      "    case '${6:ACTION_TYPE_2}':",
      "      return { ...state, ${10:updatedState2} };",
      "    default:",
      "      return state;",
      "  }",
      "};",
      "",
      "const [${11:state}, ${12:dispatch}] = useReducer(${8:reducerName}, { ${2:key: value} });",
    ],
    "description": "Creates a dynamic useReducer hook with customizable state and action types."
  }
}
```

How to Use:

- Prefix: Type `useReducerDynamic` and press `Tab`.
- Replace the placeholders with your custom values.
- The snippet will generate a customizable `useReducer` setup. You can change the state and action types, define your action types, and update the reducer logic.

Example Usage:

When you trigger the snippet, it will allow you to dynamically name your state, action types, and reducer logic:

```
type AuthAction = { type: "LOGIN"; user: string } | { type: "LOGOUT" };

const authReducer = (state: AuthState, action: AuthAction): AuthState => {
  switch (action.type) {
    case "LOGIN":
      return { ...state, isAuthenticated: true, user: action.user };
    case "LOGOUT":
      return { ...state, isAuthenticated: false, user: null };
    default:
      return state;
  }
};

const [authState, dispatch] = useReducer(authReducer, {
  isAuthenticated: false,
  user: null,
});
```

Example Snippet: Angular Snippet Example

For Angular, let's create a simple `@Component` decorator setup with basic metadata and a template. This snippet will use the file name for the component's name.

```
{
  "Angular Component": {
    "prefix": "ngc",
    "body": [
      "import { Component } from '@angular/core';",
      "",
      "@Component({",
      "  selector: 'app-${TM_FILENAME_BASE}',",
      "  template: `",
      "    <div>",
      "      <h1>${TM_FILENAME_BASE} Component</h1>",
      "    </div>`",
      "})",
      "export class ${TM_FILENAME_BASE}Component { }"
    ],
    "description": "Creates a new Angular component"
  }
}
```


- The snippet will generate an Angular component with the file name as the component name.
- `TM_FILENAME_BASE` : The file name without the extension.

Example Snippet: Vue Component Example

For Vue.js, let's create a simple component setup with a template, script, and style block. This snippet will use the file name for the component's name.

```
{
  "Vue Component": {
    "prefix": "vuec",
    "body": [
      "<template>",
      "  <div>",
      "    <h1>${TM_FILENAME_BASE} Component</h1>",
      "  </div>",
      "</template>",
      "",
      "<script setup>",
      "import { ref } from 'vue';",
      "",
      "export default {",
      "  name: '${TM_FILENAME_BASE}Component',",
      "  data() {",
      "    return {",
      "      // your data here",
      "    };",
      "  },",
      "  methods: {",
      "    // your methods here",
      "  }",
      "};",
      "</script>",
      "",
      "<style scoped>",
      "/* your styles here */",
      "</style>"
    ],
    "description": "Creates a new Vue component"
```

How to Use:

- Prefix: Type `vuec` and press `Tab`.
- The snippet will generate a Vue component with the file name as the component name.
- `TM_FILENAME_BASE`: The file name without the extension.

Example Snippet: Node.js Express Route

For Node.js, let's create a simple Express route setup with a basic GET request handler. This snippet will use the file name for the route name.

```
{
  "Express Route": {
    "prefix": "express",
    "body": [
      "const express = require('express');",
      "const router = express.Router();",
      "",
      "router.get('/${TM_FILENAME_BASE}', (req, res) => {",
      "  res.send('Hello from ${TM_FILENAME_BASE} route!');",
      "});",
      "",
      "module.exports = router;"
    ],
    "description": "Creates a new Express route"
  }
}
```

How to Use:

- Prefix: Type `express` and press `Tab`.
- The snippet will generate an Express route with the file name as the route name.
- `TM_FILENAME_BASE`: The file name without the extension.

Conclusion

Snippets are a powerful feature in VS Code that can help you write code faster and more efficiently. By creating your own snippets or using existing ones, you can

any other language or framework, snippets can save you time and effort by providing code templates that you can customize and reuse.

For more information on creating and using snippets in VS Code, check out the official documentation: [User-defined Snippets](#).

Happy coding!

Top comments (1)



Peter Vivo • Oct 6 '24



Something happen with dev.to all codebox strange empty lines inserted!
Comment in code near useless in this format. Not your fault

[Code of Conduct](#) • [Report abuse](#)

DEV Community



[Visualizing Promises and Async/Await](#) 🧠



Serif COLAKEL

Experienced in modern frontend (React, Angular, Vue) and backend (Node.js, NestJS and Go) tech. Passionate about clean code, architecture, and sharing knowledge through writing and documentation.

LOCATION

Ankara, Turkey

EDUCATION

Firat University

WORK

Software Engineer

JOINED

Jul 2, 2022

More from [Serif COLAKEL](#)



Circuit Breaker Patterns in Go Microservices

[#webdev](#) [#programming](#) [#go](#)

Building Resilient Go Services: Context, Graceful Shutdown, and Retry/Timeout Patterns

[#programming](#) [#go](#) [#backend](#) [#architecture](#)

Go Concurrency Mastery: Preventing Goroutine Leaks with Context, Timeout & Cancellation Best Practices

[#programming](#) [#productivity](#) [#go](#) [#beginners](#)